Sheetal Chaudhari

System call

Practice Questions on fork()

Predict the output of the following program-

1) int main(){

```
    fork();
    printf("hello");
}
```

2) int main()

```
{
  fork();
  fork();
  fork();
 printf("hello");
}
```

3)
```
int main()
{
    if(fork()==0)
      printf(“hello from child”);
    else
      printf(“hello from parent”);
}
```

4)
```
int main()
{
    int x = 1;

    if (fork() == 0)
      printf("Child has x = %d\n", ++x);
    else
      printf("Parent has x = %d\n", --x);
}
```

5)int main( )
{
  for (i = 0; i < n; i++)
      fork();
}
How many child process will be created?

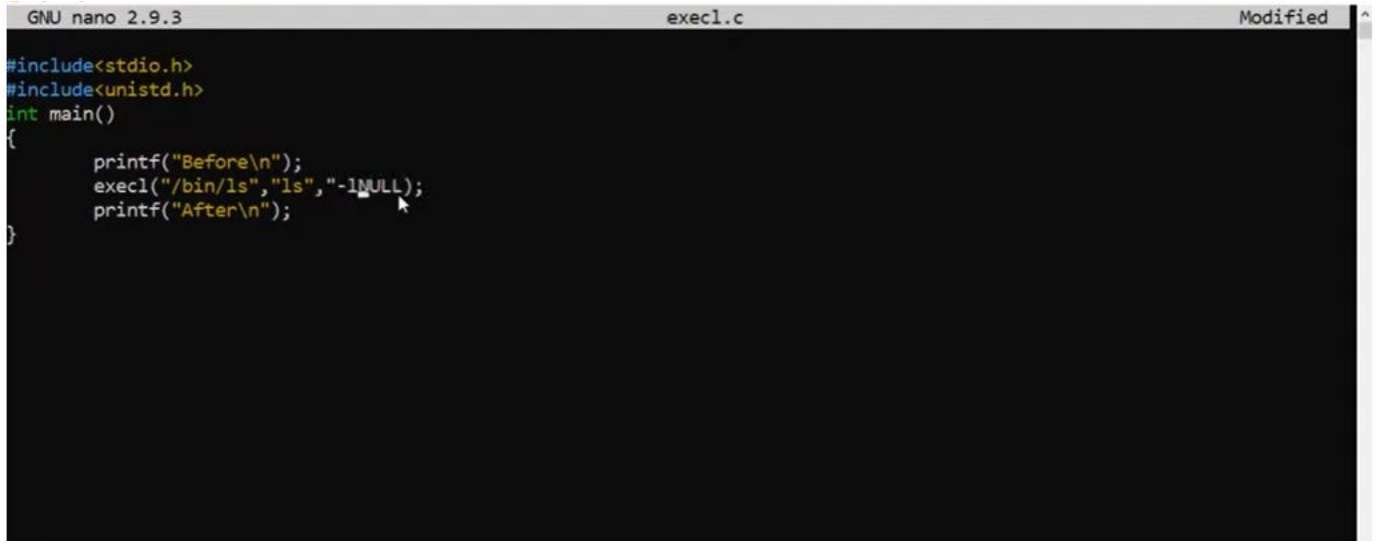
6)int main()
  {
    if (fork() || fork())
          fork();
    printf("1 ");
    return 0;
  }

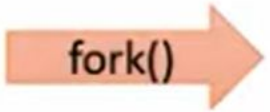
7)int main()

```
{
    if (fork() && (!fork())) {
        if (fork() || fork()) {
            fork();
        }
    }
    printf("2 ");
    return 0;
}
```

```
int main()
{
printf("before");
execl("/bin/ls","ls","-l",NULL);
printf("After");
}
```

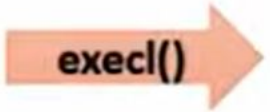
ls


GNU nano 2.9.3
execl.c
Modified
#include<stdio.h>
#include<unistd.h>
int main()
{
printf("Before\n");
execl("/bin/ls","ls","-l",NULL);
printf("After\n");
}



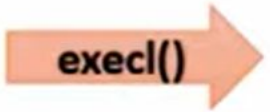

```
int main()
{
pid n;
n=fork ();
if(n==0)
{
printf(“child”);
}
else
{
printf(“Parent”);
}
```

```
int main()
{
pid n;
n=fork ();
if(n==0)
{
printf(“child”);
}
else
{
printf(“Parent”);
}
```




```
int main()
{
pid n;
n=fork ();
if(n==0)
{
printf("child");
}
else
{
printf("Parent");
}
```

Different Code from the parent




```
int main()
{
pid n;
n=fork ();
if(n==0)
{
printf("child");
}
else
{
printf("Parent");
}
```

```
int main()
{
pid n;
n=fork ();
if(n==0)
{
execl("/bin/ps","ps",NULL)
}
else
{
printf("Parent");
}
```

```c
#include<unistd.h>
#include<sys/types.h>
#include<stdio.h>
#include<sys/wait.h>
int main()
{
        pid_t q;
        q=fork();
        if(q==0)//child
        {
                printf("I am child having ID %d\n",getpid());
        }
        else{//parent
                wait(NULL);
                printf("I am parent having id %d\n",getpid());
        }
}
```

```
baljit@baljit:~/cse325/Process$ nano execl_fork.c
baljit@baljit:~/cse325/Process$ gcc execl_fork.c
baljit@baljit:~/cse325/Process$ ./a.out
I am child having ID 321
I am parent having id 320
```

Process Termination

- A process terminates when it finishes executing its final statement and ask operating system to delete it by using the exit() system call.
- At that point process may return status value to its parent process(via wait() system call).
- Operating system deallocates all the resources of the process including physical and virtual memory ,open files etc.
- A parent process can terminate the child process.

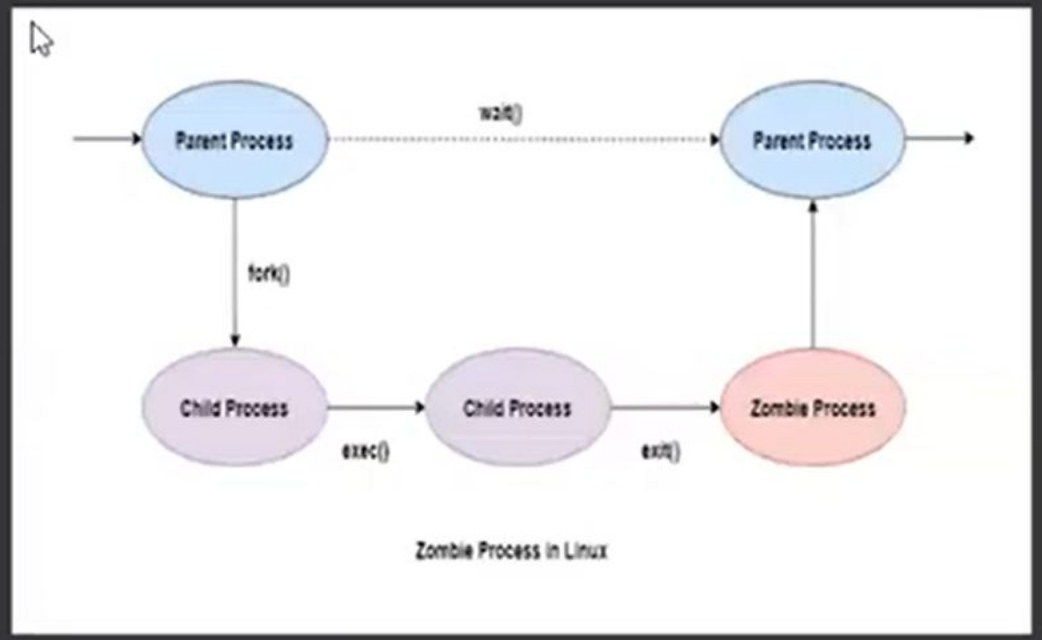
A parent may terminate the child process for the following reasons-

- The child has exceeded its usage of some resources (to determine this ,the parent must have a mechanism to inspect the state of its children.)
- The parent is exiting and operating system does not allow a child to continue if parent terminates.
- The task assigned to the child is no longer required.

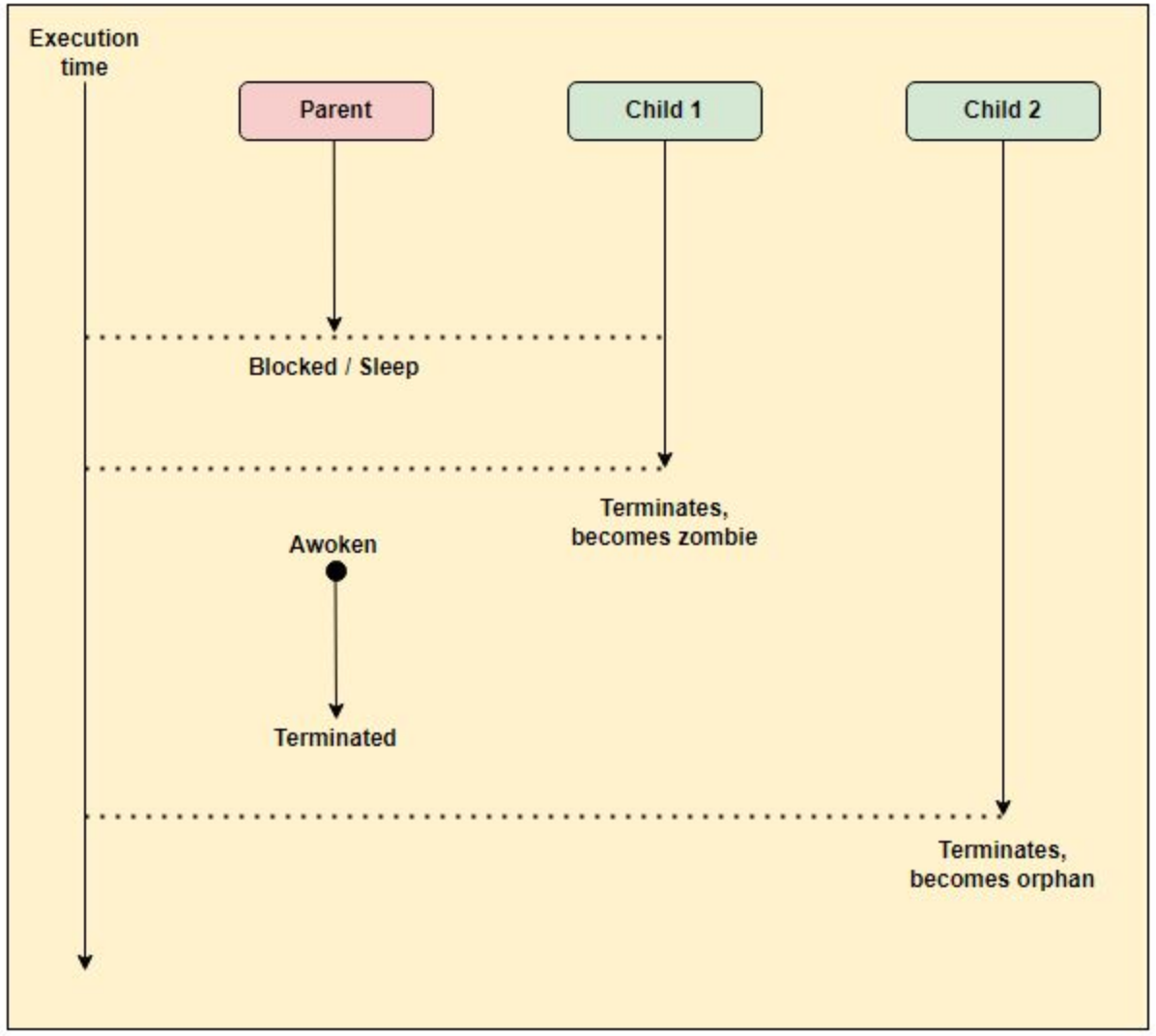
Some system do not allow a child to exist if its parent has terminated i.e. if process terminates then all its children must also be terminated. This is known as cascading termination.

Zombie Process-

- A process which has terminated but whose parent has not yet called wait(),is known as zombie process.
- Once the parent calls wait(),the process identifier of the zombie process and its entry in the process table is released.
- A child process always first becomes a zombie before being removed from the process table.



Zombie Process in Linux



Zombie and orphan process life cycle

When Child 1 terminates, we see that the parent process was asleep, so it did not collect the exit status of the child process, and the process became a zombie. So a process whose parent fails to collect its exit status upon termination becomes a zombie.

When Child 2 terminates, we see that its parent had already terminated before, so it became an orphan process, as the name states. This shows that if a process's parent terminated before the process, that process becomes an orphan.

Orphan process-

- A process whose parent process no more exists i.e. either finished or terminated without waiting for its child process to terminate is called an orphan process.
- The init process is assigned as the new parent to orphan process.
- The init process periodically invokes wait(),thereby allowing the exit status of any orphaned peocess to be collected and releasing orphan's process identifier and process table entry.

```
int main()
{
  int pid = fork();
  if (pid > 0)
     printf("in parent process");
  else if (pid == 0)
   {
      sleep(30);
      printf("in child process");
   }
}
```

```
// A C program to demonstrate Zombie Process.
// Child becomes Zombie as parent is sleeping
// when child process exits.
#include <stdlib.h>
#include <sys/types.h>
#include <unistd.h>
int main()
{
        // Fork returns process id
        // in parent process
        pid_t child_pid = fork();

        // Parent process
        if (child_pid > 0)
                sleep(50);

        // Child process
        else
                {
          printf("Pid = %d",getpid());
          printf("\n parent id = %d  ", getppid());
        exit(0);
        }
        return 0;
}
// $ gcc z1.c
/*  ./a.out &
[3] 6750
sheetal@FA-IT-Sheetal-C:~$ Pid = 6751
 parent id = 6750
sheetal@FA-IT-Sheetal-C:~$ ps
  PID TTY          TIME CMD
 4723 pts/1    00:00:00 bash
 6509 pts/1    00:07:04 a.out
 6510 pts/1    00:00:00 a.out <defunct>
 6750 pts/1    00:00:00 a.out
 6751 pts/1    00:00:00 a.out <defunct>
 6757 pts/1    00:00:00 ps
*/
```

```
// A C program to demonstrate working of orphan process

#include<stdio.h>
#include<unistd.h>
#include<sys/wait.h>
#include<sys/types.h>

int main()
{
        int i;
        int pid = fork();

        if (pid == 0)
        {
        sleep(20);
        printf("hi i m child process\n");
                printf("child id= %d",getpid());
        printf("\n my parent id is= %d ",getppid());
        }
        else
        {
                printf("I am Parent\n");
        printf("my  id is = %d",getpid());
        printf("\n my parent id is= %d ",getppid());

        }
}
/*  gcc z2.c
sheetal@FA-IT-Sheetal-C:~$ ./a.out &
[3] 7429
sheetal@FA-IT-Sheetal-C:~$ I am Parent
my  id is = 7429
 my parent id is= 4723 hi i m child process
child id= 7430
 my parent id is= 1567


*/
```